



## MODUL PERKULIAHAN

# OPTIMALISASI & OTOMASI

Kode Mata kuliah P132520004

## Modul

### Otomatisasi Monitoring: Tier Alarm,

#### Abstrak

Pertemuan ini memperluas monitoring rule-based (Pertemuan 12) menjadi arsitektur alarm management

Disusun Oleh :  
**Dedik Romahadi, ST., M.Sc**

#### Sub - CPMK

Sub-CPMK 4.3 – Mampu mengevaluasi dampak teknologi 4.0 terhadap kinerja dan kualitas

 **Modul Interaktif:** <https://dedik-romahadi.github.io/Mechanical-Engineering-Courses/Optimalisasi-dan-Automasi/Modul/Modul-12.html>. Pada layar masuk, pilih tombol “ Mode Preview (Tanpa Login)” untuk melihat modul lengkap (animasi, kalkulator interaktif, editor kode Python langsung di browser) tanpa perlu akun. Soal pada Mode Preview bersifat tampilan saja; jawaban benar/salah dan poin tidak aktif.

## PENDAHULUAN

Pertemuan 12 membahas fondasi monitoring: sensor, akuisisi, threshold ISO 10816, dan hysteresis. Namun sistem monitoring produksi sungguhan menghadapi tiga masalah tambahan: keputusan biner OK/FAIL terlalu kasar, sistem tanpa memori tidak bisa membedakan spike sesaat dari degradasi nyata, dan terlalu banyak alarm justru membanjiri operator (alarm flood). Pertemuan ketiga belas ini membahas solusinya: tier alarm bertingkat, finite state machine (FSM) dengan memori, dan standar ANSI/ISA 18.2 untuk manajemen alarm.

### Posisi Pertemuan 13 dalam Alur Mata Kuliah Optimalisasi & Otomasi



*Gambar 1. Posisi Pertemuan 13 (Tier Alarm, FSM & ANSI/ISA 18.2) dalam alur mata kuliah, melengkapi monitoring Pertemuan 12 sebelum masuk ke machine learning Pertemuan 14-15.*

Gambar 1 menunjukkan posisi Pertemuan 13 dalam alur mata kuliah.

### Capaian Pembelajaran (Sub-CPMK)

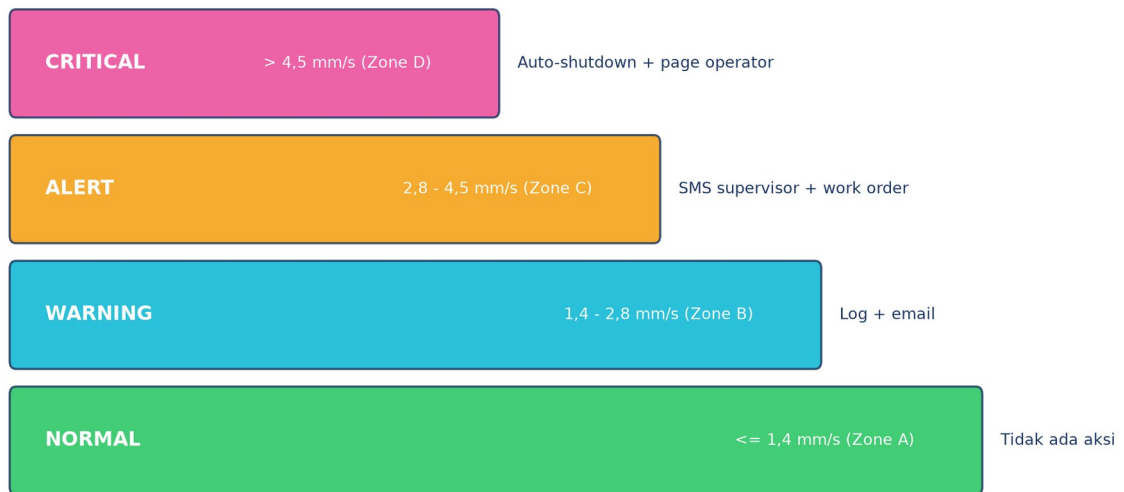
- **Sub-CPMK 4.3:** Mampu mengevaluasi dampak teknologi 4.0 terhadap kinerja dan kualitas, yaitu mengevaluasi efektivitas arsitektur tier alarm dan FSM terhadap metrik kinerja sistem (availability, MTBF/MTTR, OEE) dan kualitas manajemen alarm (ANSI/ISA 18.2) (CPMK 4).
- Setelah pertemuan ini mahasiswa dapat: merancang arsitektur tier alarm 4-level; membangun finite state machine dengan debouncing; menerapkan prinsip ANSI/ISA 18.2 untuk mencegah alarm flood; serta menghitung metrik keandalan sistem (availability, OEE, redundansi).

## TIER ALARM ARCHITECTURE: 4-LEVEL ESCALATION DARI NORMAL HINGGA CRITICAL

*4 tier (N/W/A/C)   Visual color & sound   Response SLA per tier   Auto-escalation rules   Audit log dengan timestamp*

**Empat Level Eskalasi:** Alih-alih keputusan biner, sistem tier alarm mendefinisikan 4 level: Normal (tidak ada aksi), Warning/Low (log dan email), Alert/Medium (SMS supervisor dan work order), dan Critical/High (auto-shutdown, page operator, eskalasi manajemen).

**Arsitektur Tier Alarm 4-Level: Normal → Warning → Alert → Critical**



*Gambar 2. Arsitektur tier alarm 4-level: Normal, Warning, Alert, Critical, dengan threshold ISO 10816-3 dan aksi operator masing-masing.*

Gambar 2 menunjukkan keempat tier beserta threshold dan aksi operator yang berbeda di tiap level, konsisten dengan zona ISO 10816-3 yang dibahas di Pertemuan 12.

*Tabel 1. Tier alarm, threshold, warna HMI, aksi operator, dan aturan eskalasi berbasis persistence timer.*

| Tier     | Threshold (RMS mm/s) | Color (HMI) | Action Operator      | Eskalasi (Persist)   |
|----------|----------------------|-------------|----------------------|----------------------|
| Normal   | <= 1,4 (Zone A)      | Hijau       | Tidak ada            | -                    |
| Warning  | 1,4 - 2,8 (Zone B)   | Kuning      | Log + email          | 5 menit -> Alert     |
| Alert    | 2,8 - 4,5 (Zone C)   | Oranye      | SMS supervisor       | 10 menit -> Critical |
| Critical | > 4,5 (Zone D)       | Merah       | Auto-shutdown + page | Immediate            |

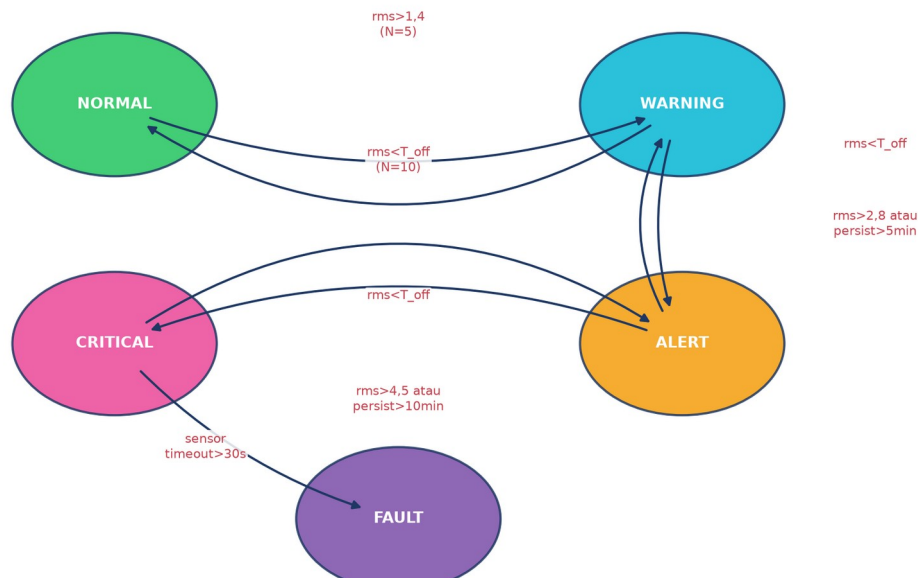
Tabel 1 merangkum keempat tier alarm beserta threshold, warna HMI standar, aksi operator, dan aturan eskalasi berbasis persistence timer.

**Contoh: Chattering Demo.** Studi kasus: RMS=2,79 mm/s beresilasi tepat di sekitar  $T_{\text{warn}}=2,8$  dengan noise  $\pm 0,05$  menghasilkan hingga 600 alert/menit tanpa hysteresis; dengan hysteresis ( $T_{\text{on}}=2,8$ ,  $T_{\text{off}}=2,6$ , margin 0,2), chattering ini hilang sepenuhnya.

## FINITE STATE MACHINE (FSM): MEMORI & TRANSITION LOGIC

**Formalisme FSM:** FSM diformalkan sebagai 5-tuple ( $S, s_0, \text{Sigma}, \text{delta}, F$ ):  $S=\{\text{NORMAL, WARNING, ALERT, CRITICAL, FAULT}\}$ ,  $s_0=\text{NORMAL}$ , Sigma=pembacaan sensor dan event timer, delta=aturan transisi. Moore machine menghasilkan output hanya berdasarkan state saat ini  $f(s)$ ; Mealy machine menghasilkan output berdasarkan state dan input  $f(s, \text{sigma})$ , lebih ekspresif tetapi lebih kompleks.

Finite State Machine Monitoring: 5 State, Debouncing & Hysteresis



Gambar 3. Diagram state FSM monitoring: 5 state (NORMAL, WARNING, ALERT, CRITICAL, FAULT) dengan transisi berbasis threshold, hysteresis, debouncing, dan sensor timeout.

Gambar 3 menunjukkan diagram lengkap FSM monitoring dengan 5 state; transisi naik (eskalasi) membutuhkan threshold terlampaui ATAU persistence timer terlampaui, sedangkan transisi turun (recovery) membutuhkan sinyal berada di bawah  $T_{\text{off}}$  (hysteresis) selama beberapa sample berturut-turut.

**Moore vs Mealy dalam Implementasi Praktis:** Pemilihan antara Moore dan Mealy machine untuk implementasi FSM monitoring berpengaruh pada bagaimana aksi (seperti pengiriman notifikasi) dipetakan terhadap state: pada Moore machine, aksi hanya bergantung pada state tujuan sehingga transisi WARNING → ALERT dan transisi langsung NORMAL → ALERT (melewati WARNING akibat kenaikan mendadak) akan memicu aksi

ALERT yang identik. Pada Mealy machine, aksi dapat dibedakan berdasarkan transisi spesifik yang terjadi, memungkinkan sistem mengirim notifikasi berbeda untuk "eskalasi bertahap terdeteksi dini" versus "lonjakan mendadak terdeteksi", informasi diagnostik tambahan yang berharga bagi tim maintenance dalam menentukan urgensi respons.

Tabel 2. Lima transisi FSM monitoring: kondisi pemicu, aksi on\_entry, dan jumlah debounce sample yang dibutuhkan.

| Transition        | Trigger Condition                              | Action (on_entry)     | Debounce N |
|-------------------|------------------------------------------------|-----------------------|------------|
| NORMAL -> WARNING | rms > T_warn (1,4)                             | Log + email           | 5 sample   |
| WARNING -> ALERT  | rms > T_alert (2,8) atau persist > 5 menit     | SMS supervisor        | 3 sample   |
| ALERT -> CRITICAL | rms > T_crit (4,5) atau persist > 10 menit     | Auto-shutdown + page  | 2 sample   |
| CRITICAL -> FAULT | sensor timeout > 30 detik                      | Mark sensor as faulty | 1 timeout  |
| Any -> NORMAL     | rms < T_off (hysteresis) dan persist > 1 menit | Log recovery          | 10 sample  |

Tabel 2 merangkum lima transisi utama FSM beserta kondisi pemicu, aksi yang dijalankan, dan jumlah sample debounce yang dibutuhkan sebelum transisi disahkan.

**Peringatan, Trap FSM & Debouncing:** Tujuh jebakan FSM dan debouncing yang sering diabaikan: state explosion (terlalu banyak state membuat sistem sulit dianalisis); race condition antar-thread yang membaca/menulis state bersamaan; stuck state karena bug logic; debounce N yang terlalu besar menyebabkan deteksi lambat; heartbeat/watchdog timer yang terlupa (sensor mati dianggap tetap "NORMAL"); audit log yang membengkak tanpa retention policy; serta tidak adanya diagram state formal sebagai dokumentasi.

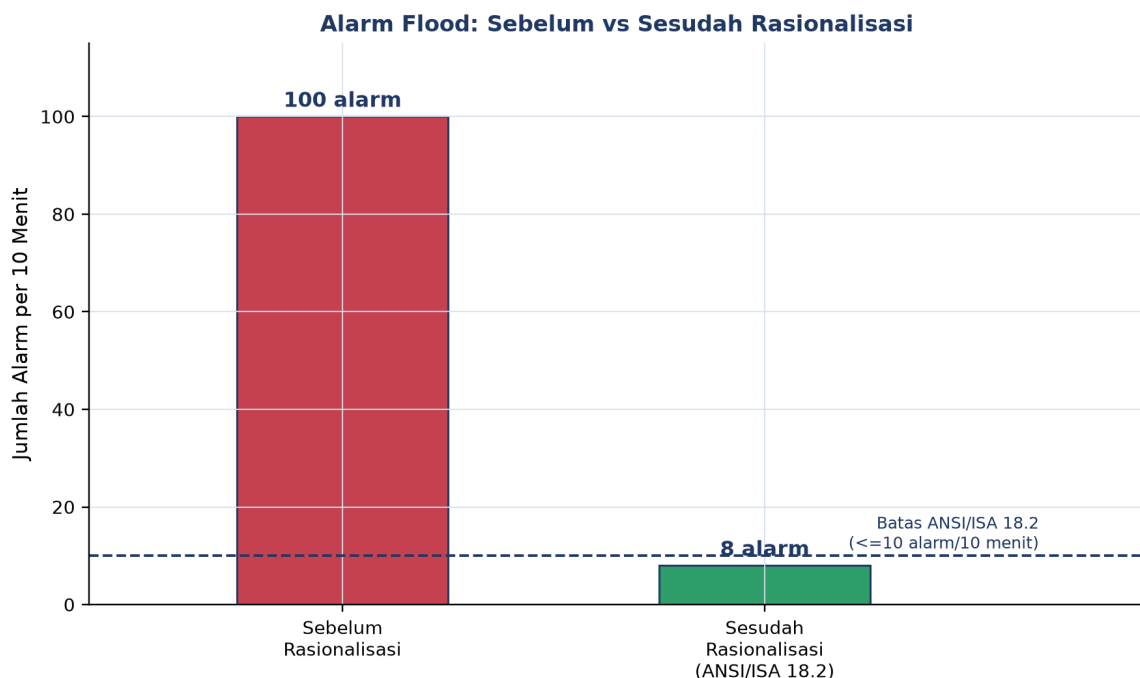
## ANSI/ISA 18.2: ALARM MANAGEMENT STANDARD

**Latar Belakang & Lifecycle:** ANSI/ISA-18.2-2016 mendefinisikan lifecycle manajemen alarm 7 fase, dari identifikasi hingga audit berkala. Standar ini lahir sebagian dari pembelajaran insiden industri seperti ledakan kilang BP Texas City (2005) yang menewaskan 15 orang, di mana alarm flood berkontribusi pada kegagalan operator merespons kondisi kritis tepat waktu.

- **Alarm Rationalization:** memastikan setiap alarm unik, benar-benar memerlukan respons operator, serta memiliki prioritas dan setpoint yang terdokumentasi; alarm tanpa aksi jelas dihapus dari sistem.

- **Alarm Shelving:** menunda/menyembunyikan sementara alarm yang sudah diketahui (operator-initiated, auto-unshelve setelah timeout) agar tidak menambah beban alarm flood.
- **Suppression Rules (Parent-Child):** menekan alarm turunan (child) ketika alarm induk (parent) sudah aktif, karena keduanya disebabkan oleh akar masalah yang sama.

**Target Alarm Rate:** Target ANSI/ISA 18.2: kondisi steady-state maksimal 1 alarm per 10 menit per operator; dalam 10 menit pertama pasca-gangguan, maksimal 10 alarm masih dapat diterima. Melebihi ini disebut alarm flood, memicu alarm fatigue yang membuat operator cenderung mengabaikan semua alarm, termasuk yang genuinely kritis.



Gambar 4. Perbandingan jumlah alarm per 10 menit sebelum dan sesudah alarm rationalization; batas ANSI/ISA 18.2 adalah 10 alarm per 10 menit.

Gambar 4 menunjukkan dampak alarm rationalization: dari 100 alarm per 10 menit (jauh melampaui batas alarm flood) menjadi 8 alarm per 10 menit (di bawah batas ANSI/ISA 18.2), tercapai lewat penghapusan alarm duplikat dan penerapan suppression rules.

### METRIK KEANDALAN SISTEM: AVAILABILITY, OEE & REDUNDANSI

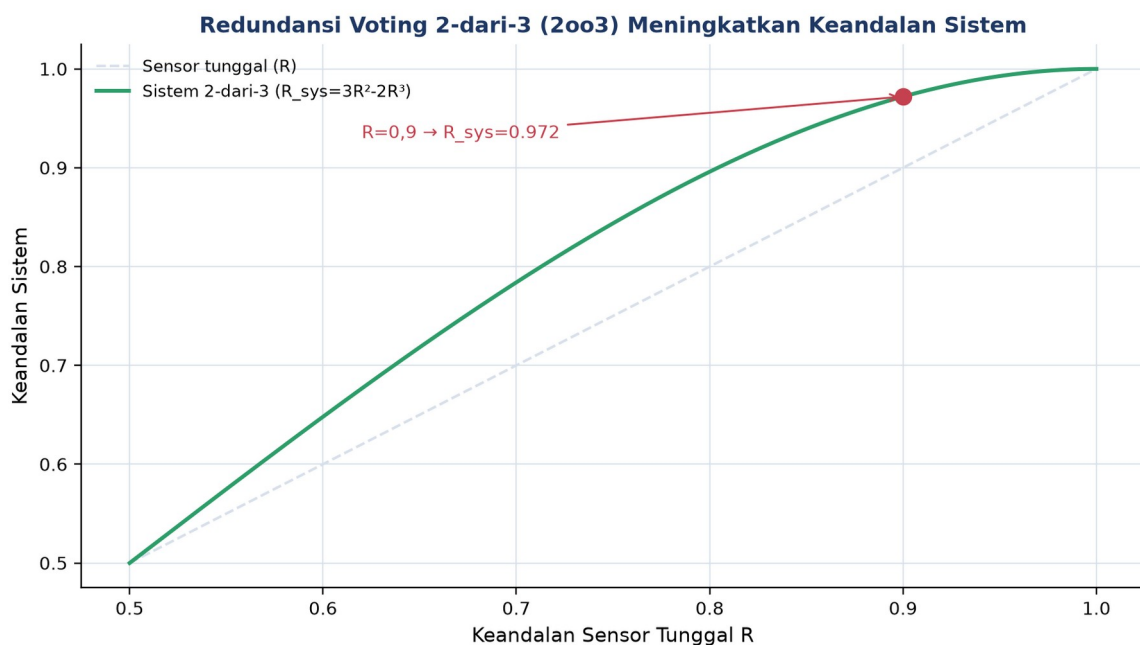
$$\text{Availability} = \text{MTBF} / (\text{MTBF} + \text{MTTR}) \quad \text{MTBF} = \text{total jam operasi} / \text{jumlah kegagalan}$$

$$\text{OEE} = \text{Availability} \times \text{Performance} \times \text{Quality} \quad 2003: R_{\text{sys}} = 3R^2 - 2R^3$$



**Availability & MTBF/MTTR:** Availability (ketersediaan) mengukur proporsi waktu mesin dapat beroperasi normal, dihitung dari Mean Time Between Failures (MTBF) dan Mean Time To Repair (MTTR).

**Contoh Perhitungan Availability:** Sebagai contoh perhitungan konkret: mesin dengan MTBF=720 jam (30 hari operasi rata-rata antar kegagalan) dan MTTR=8 jam (waktu perbaikan rata-rata) menghasilkan  $\text{Availability} = 720 / (720 + 8) = 98,9\%$ , angka yang tampak tinggi namun setara dengan sekitar 96 jam downtime per tahun, cukup signifikan untuk lini produksi kontinu bernilai tinggi. Menurunkan MTTR (respons perbaikan lebih cepat, biasanya via monitoring kondisi yang memberi peringatan dini) umumnya jauh lebih murah dan lebih cepat diimplementasikan dibanding menaikkan MTBF (yang membutuhkan overhaul desain atau penggantian komponen berkualitas lebih tinggi).

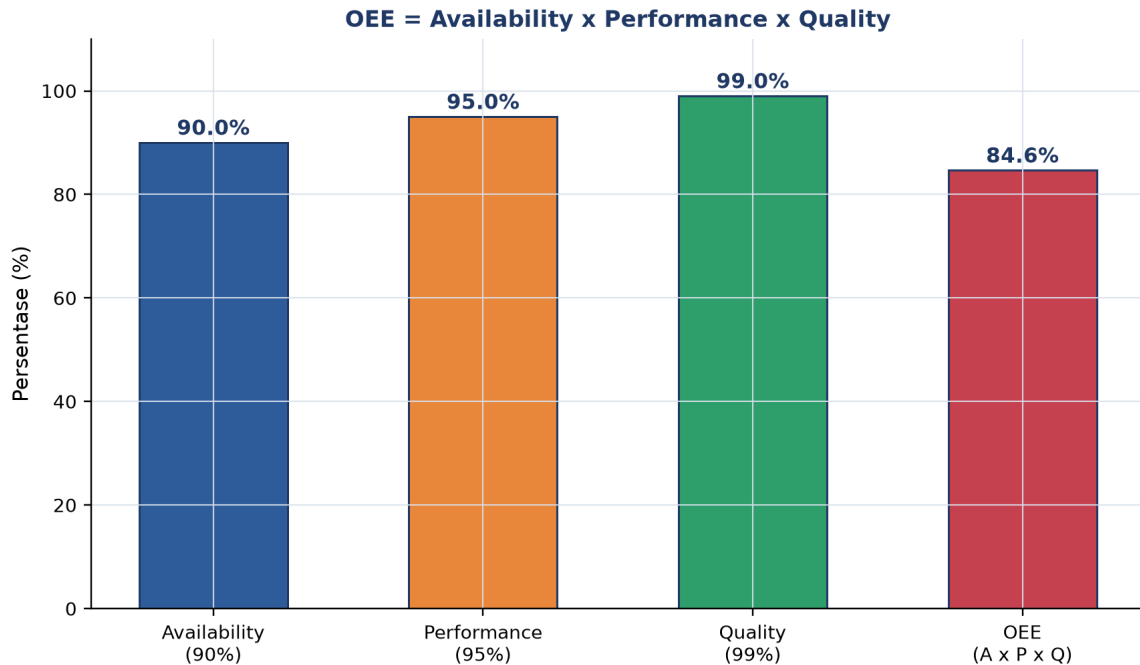


Gambar 5. Keandalan sistem redundan 2-dari-3 (2oo3) dibanding sensor tunggal:  $R_{\text{sys}} = 3R^2 - 2R^3$  selalu lebih tinggi dari  $R$  untuk  $R > 0,5$ .

Gambar 5 menunjukkan bagaimana skema voting 2-dari-3 (sistem trip hanya bila minimal 2 dari 3 sensor sepakat) meningkatkan keandalan sistem secara signifikan dibanding mengandalkan satu sensor saja; untuk keandalan sensor tunggal  $R=0,9$ , keandalan sistem 2oo3 naik menjadi 0,972, sekaligus toleran terhadap kegagalan 1 sensor tanpa menyebabkan false trip.

**Mengapa OEE sebagai Metrik Komposit:** OEE dipilih sebagai metrik komposit tunggal karena sifat perkaliannya memaksa tim maintenance mengoptimalkan seluruh rantai produktivitas secara seimbang, bukan hanya salah satu aspek. Pabrik yang hanya berfokus meningkatkan Availability (misalnya lewat preventive maintenance agresif) tanpa

memperhatikan Performance (kecepatan aktual dibanding kecepatan ideal) atau Quality (proporsi produk sesuai standar) tetap akan memiliki OEE rendah meski mesinnya jarang berhenti, karena downtime bukan satu-satunya sumber kerugian produktivitas.



Gambar 6. Overall Equipment Effectiveness (OEE) sebagai perkalian tiga faktor: Availability, Performance, dan Quality.

Gambar 6 mengilustrasikan bagaimana  $OEE = Availability \times Performance \times Quality$ ; meskipun setiap faktor individual terlihat tinggi (90%, 95%, 99%), OEE gabungan turun menjadi 84,6% karena sifat perkalian, menegaskan pentingnya optimasi ketiga faktor secara bersamaan bukan hanya salah satu.

## ANALOGI KEHIDUPAN SEHARI-HARI

- **Termostat AC (Hysteresis Klasik):** setpoint 24°C dengan hysteresis 1°C mencegah kompresor AC menyala-mati berulang; mengacu pada prinsip Schmitt trigger yang sama dengan Pertemuan 12.
- **Indikator Bensin Mobil (Tier Alarm):** indikator bensin mobil:  $\frac{1}{4}$  tank (warning), E-line dan ikon pompa (alert, sekitar 10km), buzzer dan lampu merah (critical, sekitar 5km); mobil modern seperti Tesla memakai FSM untuk logika ini.
- **Alarm Asap di Rumah (Debouncing):** standar UL 217/EN 14604 mensyaratkan konsentrasi asap terdeteksi kontinu minimal 30 detik (debouncing) sebelum alarm berbunyi, plus bunyi "beep" heartbeat baterai lemah setiap 30 detik.



- **Smartwatch Heart Rate Alarm (Multi-Threshold):** Apple Watch dan Fitbit mendeteksi HR>120bpm tanpa olahraga selama 10 menit (high) dan HR<40bpm selama 10 menit (low); FDA menyetujui deteksi AFib dengan klaim false-positive di bawah 0,5%.
- **Lampu Lalu Lintas (FSM Klasik):** lampu lalu lintas 3-state (Hijau 30 detik, Kuning 5 detik, Merah 30 detik) dengan interlock lintas-arah adalah FSM klasik paling sederhana.
- **DevOps Server Monitoring (Production-Grade):** Netflix, Google, dan AWS memakai tier CPU>80% selama 5 menit (warning ke PagerDuty/Slack), CPU>95% selama 2 menit (alert ke on-call), dan response time>5 detik (critical, auto-failover), dengan tools Prometheus+Alertmanager, Grafana, Datadog, dan New Relic.

**Pola Umum & Keterbatasan:** Tiga elemen yang sama muncul di semua analogi: threshold dengan referensi jelas, filter noise (hysteresis/debouncing), dan eskalasi bertahap. Rule-based memiliki batas skala; ketika kompleksitas sistem melebihi puluhan fitur atau ratusan mode kegagalan, pendekatan machine learning (Pertemuan 14-15) menjadi lebih tepat.

## APLIKASI RULE-BASED MONITORING DI INDUSTRI 4.0 & IOT MANUFAKTUR

- **Wind Turbine SCADA Monitoring (Vestas):** 200+ sensor per turbin mengikuti standar ISO 10816-21, memakai OSIsoft PI System dan Bachmann Webmonitor, menurunkan MTTR sebesar 30%.
- **Gedung Tinggi Smart HVAC (Siemens Building):** 10.000+ titik data pada gedung seperti Burj Khalifa dan 1WTC, tier CO2 800/1000/1500 ppm dengan hysteresis 100ppm, menghemat energi 15-25% mengikuti standar ASHRAE 90.1 dan EN 15251.
- **Power Grid Substation Monitoring (PLN, ABB):** gardu induk 150kV/500kV dengan protokol IEC 61850, rule rate-of-change (kenaikan suhu oli >5°C/menit=critical), mendeteksi 90% gangguan sebelum blackout; vendor ABB Relion, Siemens Reyrolle, Schneider MiCOM.
- **F1 Race Car Telemetry (Mercedes-AMG, McLaren):** 300+ sensor telemetri dengan latency di bawah 50 milidetik, tier suhu rem (warning 700°C, critical 850°C), mengikuti regulasi retensi data FIA.

## IMPLEMENTASI PYTHON RULE-BASED MONITORING DI JUPYTER NOTEBOOK

Empat cell berikut menerapkan konsep pertemuan ini pada studi kasus pompa sentrifugal. Lingkungan: conda env optoauto dengan paket numpy, pandas, matplotlib; notebook p13\_tier\_alarm\_fsm.ipynb.

p13\_tier\_alarm\_fsm.ipynb — Cell 3 (MonitoringFSM)

```
class MonitoringFSM:
    STATES = ['NORMAL', 'WARNING', 'ALERT',
              'CRITICAL', 'FAULT']
    THRESHOLDS = {'WARNING':1.4, 'ALERT':2.8,
                  'CRITICAL':4.5}
    HYSTERESIS = 0.2
    DEBOUNCE_N = 3

    def update(self, x, ts=None):
        target = self._target_state(x)
        # debounced up/down counter...
        if self.up_counter >= self.DEBOUNCE_N:
            self._transition(target, x, ts)
```

Gambar 7. Cuplikan class `MonitoringFSM` dengan definisi state, threshold, hysteresis, dan debounce.

Gambar 7 menunjukkan cuplikan kode inti class `MonitoringFSM`.

- **Cell 1:** fungsi `iso10816_zone()` untuk klasifikasi zona diterapkan pada sinyal degradasi 30 hari sintetis pompa 50kW.
- **Cell 2:** class `HysteresisAlarm` (`T_on/T_off`) dibandingkan `SimpleThreshold` pada sinyal RMS 500 sample; hysteresis menekan chattering secara signifikan.
- **Cell 3:** class `MonitoringFSM` lengkap: 5 state, threshold {WARNING:1,4; ALERT:2,8; CRITICAL:4,5}, hysteresis 0,2, debounce N=3, audit log ke pandas DataFrame; diuji pada sinyal 12 jam (1440 sample) dengan 3 spike noise yang berhasil difilter debouncing.
- **Cell 4:** fungsi `run_monitoring_pipeline()` reusable menggabungkan klasifikasi zona, FSM, audit log, statistik persentase waktu per state, dan dashboard 3-panel; didemonstrasikan pada simulasi 24 jam (2880 sample).

Fungsi `run_monitoring_pipeline()` pada Cell 4 dirancang reusable agar dapat langsung diadaptasi ke sensor lain hanya dengan mengganti parameter threshold dan hysteresis, tanpa perlu menulis ulang logika FSM dan audit log dari nol, mencerminkan praktik rekayasa perangkat lunak yang baik untuk sistem monitoring produksi yang harus dipasang pada puluhan hingga ratusan titik sensor berbeda.

## TUGAS PERTEMUAN 13

Tugas terdiri atas 10 soal Pilihan Ganda (1 poin), 10 soal Komputasi Easy-Medium (2 poin), dan 5 soal Komputasi Hard (4 poin), total 50 poin.

### Distribusi Bobot Tugas Pertemuan 13



Gambar 8. Distribusi 50 poin Tugas Pertemuan 13 berdasarkan tipe soal.

Gambar 8 merangkum distribusi bobotnya.

### Bagian A: Pilihan Ganda (10 soal × 1 poin)

1. Dalam lifecycle ANSI/ISA 18.2, tujuan tahap alarm rationalization adalah...

- A. Menambah sebanyak mungkin alarm agar operator selalu waspada
- B. Memastikan setiap alarm unik, benar-benar perlu respons operator, serta punya prioritas dan setpoint terdokumentasi; alarm tanpa aksi jelas dihapus
- C. Mengganti semua alarm dengan model machine learning
- D. Menonaktifkan seluruh alarm low-priority secara permanen

2. Kondisi alarm flood menurut ANSI/ISA 18.2 terjadi ketika...

- A. Operator menerima kurang dari 1 alarm per jam
- B. Semua alarm berprioritas rendah
- C. Lebih dari 10 alarm dalam 10 menit per operator, melampaui kapasitas kognitif sehingga operator cenderung mengabaikan semua alarm (alarm fatigue)
- D. Alarm muncul tepat 1 kali per 10 menit

3. Ketersediaan (availability) sebuah mesin dinyatakan sebagai...

- A.  $A = \text{MTBF} / (\text{MTBF} + \text{MTTR})$
- B.  $A = \text{MTTR} / \text{MTBF}$
- C.  $A = \text{MTBF} \times \text{MTTR}$
- D.  $A = 1 - \text{MTBF}$

**4. Skema sensor redundan 2-out-of-3 (2oo3) meningkatkan keandalan dengan cara...**

- A. Memakai 1 sensor saja agar sederhana
- B. Mengambil rata-rata semua sensor tanpa logika voting
- C. Sistem trip hanya bila ketiga sensor setuju
- D. Keputusan diambil bila minimal 2 dari 3 sensor setuju, toleran terhadap 1 sensor gagal sekaligus menahan false trip

**5. Fitur alarm shelving dalam alarm management berfungsi untuk...**

- A. Menghapus alarm secara permanen dari sistem
- B. Menunda/menyembunyikan alarm yang sudah diketahui untuk sementara (operator-initiated, auto-unshelve setelah timeout) agar tidak menambah flood
- C. Menaikkan prioritas semua alarm
- D. Mengirim alarm ke semua operator sekaligus

**6. Overall Equipment Effectiveness (OEE) dihitung sebagai...**

- A.  $OEE = Availability + Performance + Quality$
- B.  $OEE = Availability - Downtime$
- C.  $OEE = Availability \times Performance \times Quality$
- D.  $OEE = MTBF / MTTR$

**7. Penentuan prioritas alarm (ANSI/ISA 18.2) idealnya didasarkan pada...**

- A. Urutan abjad nama tag
- B. Tingkat konsekuensi (safety > environmental > economic) dan waktu yang tersedia untuk merespons
- C. Warna favorit operator
- D. Jumlah karakter pada pesan alarm

**8. Time-series database (misal TimescaleDB / InfluxDB) cocok untuk data sensor karena...**

- A. Mendukung hypertable/partisi waktu, continuous aggregate (downsampling otomatis), kompresi dan retention policy untuk volume tinggi
- B. Hanya menyimpan satu nilai terakhir
- C. Tidak mendukung query rentang waktu
- D. Memerlukan input spreadsheet manual

**9. Keunggulan utama edge computing untuk monitoring getaran di pabrik adalah...**

- A. Selalu butuh koneksi cloud agar berfungsi
- B. Menghapus kebutuhan sensor
- C. Latency lebih tinggi daripada cloud
- D. Latency rendah, hemat bandwidth (kirim fitur, bukan raw), dan tetap berjalan saat koneksi cloud putus (resilient)

10. Metrik kinerja sistem alarm (alarm system performance) yang dipantau rutin mencakup...

- A. Hanya jumlah sensor terpasang
- B. Warna dashboard
- C. Rata-rata laju alarm ter-annunciate, jumlah standing/stale alarm, dan persen waktu dalam kondisi flood
- D. Harga lisensi software

### Bagian B: Komputasi Easy-Medium (10 soal × 2 poin)

Catatan:  $\text{Availability} = \text{MTBF} / (\text{MTBF} + \text{MTTR})$ ;  $\text{MTBF}$  = total jam operasi/jumlah kegagalan;  $\text{OEE} = \text{Availability} \times \text{Performance} \times \text{Quality}$ ; redundansi 2oo3 voting:  $R_{\text{sys}} = 3R^2 - 2R^3$ ; ANSI/ISA 18.2: target  $\leq 1$  alarm/10 menit/operator, alarm flood  $\Rightarrow 10$  alarm/10 menit/operator.

**C1.** Hitung availability (persen) sebuah mesin dengan  $\text{MTBF} = 200$  jam dan  $\text{MTTR} = 8$  jam.  
Formula:  $A = \text{MTBF} / (\text{MTBF} + \text{MTTR}) \times 100$ .

- 💡 **HINT:** Availability = uptime rata-rata dibagi total siklus (uptime+repair), dikali 100.

**C2.** Target ANSI/ISA 18.2  $\leq 1$  alarm/10 menit per operator (steady state). Hitung jumlah alarm per hari maksimum yang masih compliant.

- 💡 **HINT:** 1 alarm/10 menit  $\rightarrow$  per jam, lalu dikali 24.

**C3.** Hitung OEE (persen) untuk Availability=0,90, Performance=0,95, Quality=0,99.  
Formula:  $\text{OEE} = A \times P \times Q \times 100$ .

- 💡 **HINT:** Kalikan ketiga faktor lalu dikali 100.

**C4.** Keandalan sistem 2oo3 (dua dari tiga) bila keandalan tiap sensor  $R = 0,9$ :  $R_{\text{sys}} = 3R^2 - 2R^3$ . Hitung  $R_{\text{sys}}$ .

- 💡 **HINT:** Substitusi  $R = 0,9$  ke  $3R^2 - 2R^3$ .

**C5.** Sebuah mesin mengalami 5 kegagalan dalam 10000 jam operasi. Hitung MTBF (jam) = total jam operasi / jumlah kegagalan.

- 💡 **HINT:** Bagi total jam dengan jumlah kegagalan.

**C6.** Durasi alarm aktif (menit): [2, 45, 5, 120, 30]. Hitung jumlah standing alarm (durasi > 30 menit).

- 💡 **HINT:** Hitung elemen yang nilainya lebih besar dari 30.

**C7.** Data sensor 1 Hz selama 1 jam di-downsample menjadi rata-rata per 1 menit (continuous aggregate). Hitung jumlah titik output.

- 💡 **HINT:** 3600 detik dibagi 60 detik per bucket.

**C8.** Untuk target availability 99,5% selama 1 tahun (8760 jam), hitung maksimum downtime (jam) =  $8760 \times (1 - 0,995)$ .

- 💡 **HINT:** Downtime budget = total jam  $\times$  (1 - target).

**C9.** Daftar prioritas alarm: ['L','H','C','C','M','C']. Hitung jumlah alarm berprioritas Critical ('C').

- 💡 **HINT:** Hitung kemunculan 'C' di list.

**C10.** Sebelum rasionalisasi: 500 alarm/hari; sesudah: 120 alarm/hari. Hitung pengurangan (persen) =  $(500 - 120) / 500 \times 100$ .

- 💡 **HINT:** Selisih dibagi nilai awal, dikali 100.

### Bagian C: Komputasi Hard (5 soal $\times$ 4 poin)

Setiap soal membutuhkan 20-50+ baris kode dengan algoritma lengkap dari awal, hint minimal. Jawaban benar: +4 poin. Jawaban salah (kode ada tapi hasil tidak sesuai/error): +1 poin partial. Kode kosong = 0 poin (bisa retry).

**C11 (Hard).** Availability dari event log: diberi waktu antar-kegagalan up=[100,150,200] jam dan waktu perbaikan down=[5,8,6] jam. Implementasikan perhitungan MTBF=mean(up), MTTR=mean(down), lalu print availability (persen)= $MTBF / (MTBF + MTTR) \times 100$ .

- 💡 **HINT:** Rata-ratakan up dan down, lalu terapkan formula availability.

**C12 (Hard).** Monte Carlo keandalan 2oo3: np.random.seed(42); 10000 trial, tiap trial 3 sensor dengan probabilitas gagal 0,1 (np.random.rand(10000,3)<0.1). Sistem GAGAL bila  $\geq 2$  sensor gagal. Print keandalan sistem (fraksi trial yang sistemnya OK).

- 💡 **HINT:** Hitung jumlah sensor gagal per trial; sistem OK bila gagal < 2; rata-ratakan.

**C13 (Hard).** Deteksi alarm flood (sliding window): np.random.seed(42); timestamp alarm (detik)=np.sort(np.random.uniform(0,3600,80)). Dengan window 10 menit (600 detik), hitung jumlah alarm maksimum dalam satu window (geser tiap alarm sebagai awal window).

- 💡 **HINT:** Untuk tiap alarm t, hitung alarm pada [t, t+600); ambil maksimum.

**C14 (Hard).** OEE dari time-series: np.random.seed(42);  
 $A = np.random.uniform(0.85, 0.98, 30)$ ;  $P = np.random.uniform(0.80, 0.95, 30)$ ;  
 $Q = np.random.uniform(0.95, 1.0, 30)$ . Print OEE (persen)= $mean(A) \times mean(P) \times mean(Q) \times 100$ .

- 💡 **HINT:** Rata-ratakan tiap faktor 30-hari, kalikan, dikali 100.

**C15 (Hard).** End-to-end availability 24 jam: simulasi 24 jam (sampling 30 detik, n=2880). np.random.seed(42); state\_up=np.random.rand(2880) > 0.03 (mesin down 3% waktu). Print persentase downtime (persen)= $100 \times mean(\sim state\_up)$ .

- 💡 **HINT:** Hitung fraksi sample yang state\_up False, dikali 100.

**DAFTAR PUSTAKA**

- Ul Hassan, I., Panduru, K., & Walsh, J. (2024). An in-depth study of vibration sensors for condition monitoring. *Sensors*, 24(3), 740. <https://doi.org/10.3390/s24030740>
- Romanssini, M., de Aguirre, P. C. C., Compassi-Severo, L., & Girardi, A. G. (2023). A review on vibration monitoring techniques for predictive maintenance of rotating machinery. *Eng*, 4(3), 1797–1817. <https://doi.org/10.3390/eng4030102>
- Tiboni, M., Remino, C., Bussola, R., & Amici, C. (2022). A review on vibration-based condition monitoring of rotating machinery. *Applied Sciences*, 12(3), 972. <https://doi.org/10.3390/app12030972>
- Nguyen, T. T., Phi, B. P., Tran, V. T., Nguyen, V.-T., & Nguyen, V. T. T. (2025). Three-stage optimization of surface finish in WEDM of D2 tool steel via Taguchi design and ANOVA analysis. *Metals*, 15(6), 682. <https://doi.org/10.3390/met15060682>
- Bayá, G., Canale, E., Nesmachnow, S., De Sousa, A., & Sartor, P. (2022). Production optimization in a grain facility through mixed-integer linear programming. *Applied Sciences*, 12(16), 8212. <https://doi.org/10.3390/app12168212>
- Zhang, M., Guo, J., Li, X., & Jin, R. (2020). Data-driven anomaly detection approach for time-series streaming data. *Sensors*, 20(19), 5646. <https://doi.org/10.3390/s20195646>